2025

Bettr.

Documentation

Hugo Montes Fernández

Bettr - Technical Documentation

Intermodular Project DAM - Course 2025/2026

Author: Hugo Montes
Cycle: Multiplatform Application Development (DAM)
Course: 2025/2026

Table of Contents

1. System Architecture
2. API
3. Database
4. Web Application
5. Mobile Application
6. Desktop Application
7. Security
8. Deployment
9. Testing
10. Usage and Maintenance Guide
11. Future Improvements

---

1. System Architecture

1.1 General Overview

Bettr is a social habit tracking application that allows users to create and share their daily h abits with others. The system consists of four main components that communicate through a REST API:

```
┌──────────────┐   ┌──────────────┐   ┌──────────────┐
│ Android App  │   │Desktop App   │   │ Web App      │
│  (Kotlin)    │   │  (C#)        │   │  (PHP)       │
└──────┬───────┘   └──────┬───────┘   └──────┬───────┘
       │                  │                  │
       └──────────────────┴──────────────────┘
                          │
                          ▼
                  ┌──────────────┐
                  │  REST API    │
                  │  (Java)      │
                  └──────┬───────┘
                         │
                         ▼
                  ┌──────────────┐
                  │  Supabase    │
                  │(PostgreSQL)  │
                  └──────────────┘
```

1.2 Main Technologies

Component	Technology	Description
REST API	Java + JAX-RS (Jersey)	Backend handling all requests
Database	PostgreSQL (Supabase)	Cloud database
Android App	Kotlin/Java	Native mobile application
Desktop App	C# + Windows Forms	Windows desktop application
Web App	PHP	Responsive web application
API Build	Maven	Dependency manager
Android Build	Gradle	Dependency manager

---

2. API

2.1 Endpoint List

Users

Endpoint	Method	Description
`/users`	POST	Register new user
`/users/{username}`	GET	Get user by username
`/users/{id}`	GET	Get user by ID
`/users`	GET	Search users
`/users/{id}/profile`	POST	Update profile

Habits

Endpoint	Method	Description
`/habits`	POST	Create new habit
`/habits/feed/{userId}`	GET	Get habit feed
`/habits/user/{userId}`	GET	Get user's habits
`/habits/{id}/like/{userId}`	POST	Like a habit
`/habits/{id}/like/{userId}`	DELETE	Unlike a habit

Followers

Endpoint	Method	Description
`/users/follow/{followerId}/{followingId}`	POST	Follow a user

Endpoint	Method	Description
`/users/unfollow/{followerId}/{followingId}`	DELETE	Unfollow
`/users/{userId}/followers`	GET	Get followers
`/users/{userId}/following`	GET	Get following

2.2 Authentication

The API uses session token authentication. Users log in by sending their credentials (usern ame/password) and the API returns the user data along with a session token.

Note: Passwords are stored hashed using **BCrypt** for enhanced security.

2.3 Request and Response Examples

Register User

Request:

```
POST https://bettr-g5yv.onrender.com/rest/users
Content-Type: application/json

{
  "username": "juanito",
  "email": "juan@example.com",
  "password": "micontraseña"
}
```

Response:

```
{
  "id": 1,
```

```
  "username": "juanito",
  "email": "juan@example.com",
  "avatar": "default.png",
  "description": ""
}
```

Get Feed

Request:

```
GET https://bettr-g5yv.onrender.com/rest/habits/feed/1
```

Response:

```
[
  {
    "id": 5,
    "userId": 2,
    "username": "mariano",
    "description": "Hoy he salido a correr 5km",
    "image": "habit_123.jpg",
    "type": "exercise",
    "likes": 10,
    "createdAt": "2026-03-28T10:30:00Z"
  }
]
```

Create Habit

Request:

```
POST https://bettr-g5yv.onrender.com/rest/habits
Content-Type: application/json
```

```
{
  "userId": 1,
  "description": "Nuevo hábito diario",
  "type": "health"
}
```

---

3. Database

3.1 Entity-Relationship Diagram

```
┌──────────────────┐      ┌──────────────────┐      ┌──────────────────┐
│    users    │      │   habits    │      │ followers   │
├──────────────────┤      ├──────────────────┤      ├──────────────────┤
│ id (PK)     │◄─────────│ user_id (FK)│      │follower_id  │
│ username    │      │ id (PK)     │      │ following_id │
│ email       │      │ description │      │ (PK)        │
│ password    │      │ image       │      └──────────────────┘
│ avatar      │      │ type        │
│ description │      │ created_at  │
│ created_at  │      │ user_id     │
└──────────────────┘      └──────────┬───────┘
                   │
              ┌────────▼────────┐
              │ habit_likes │
              ├──────────────────┤
              │ habit_id (FK)│
              │ user_id (FK) │
              └──────────────────┘
```

3.2 Table and Field Descriptions

Table: users

Field	Type	Description
id	INTEGER	Unique identifier
username	VARCHAR(50)	Unique username
email	VARCHAR(100)	Unique email
password	VARCHAR(255)	Hashed password
avatar	VARCHAR(255)	Profile image path
description	TEXT	User biography
created_at	TIMESTAMP	Registration date

Table: habits

Field	Type	Description
id	INTEGER	Unique identifier
user_id	INTEGER	FK to users
description	TEXT	Habit description
image	VARCHAR(255)	Attached image (optional)
type	VARCHAR(50)	Habit category
created_at	TIMESTAMP	Creation date

Table: followers

Field	Type	Description
follower_id	INTEGER	FK to users (follower)
following_id	INTEGER	FK to users (followed)

Table: habit_likes

Field	Type	Description
habit_id	INTEGER	FK to habits
user_id	INTEGER	FK to users

3.3 Important Rules

- **Usernames and emails** must be unique in the `users` table
- **Password** is stored using **BCrypt** for security
- **follower_id and following_id** form a composite primary key in `followers`
- **habit_likes** uses a composite primary key (habit_id, user_id) to prevent duplicates

3.4 Migration Management

The database is managed through **Supabase**, which provides a visual interface for creating and modifying tables. Migrations are performed manually through the Supabase panel or by executing SQL scripts directly in the PostgreSQL console.

---

4. Web Application

4.1 Technology Used

- **PHP 8.x** - Server-side programming language
- **MVC (Model-View-Controller)** - Architectural pattern
- **HTML/CSS/JavaScript** - Frontend
- **cURL** - For making requests to the API

4.2 Project Structure

```
Bettr_Web_App/
├── public/
│   ├── index.php         # Entry point
│   ├── login.php         # Login page
│   ├── registro.php      # Registration page
│   ├── dashboard.php     # Main dashboard
│   └── api_proxy.php     # Bridge to the API
├── app/
│   ├── config/
│   │   └── database.php  # Database configuration
│   ├── controllers/      # Controllers
│   ├── models/           # Data models
│   ├── services/
│   │   └── AuthService.php # Authentication service
│   └── helpers/
│       └── session.php   # Session management
└── views/                # Views (optional)
```

---

5. Mobile Application

5.1 Technology Used

- **Kotlin** - Main language
- **Java** - Legacy code compatibility
- **Android Studio** - Development IDE
- **Gradle** - Build system
- **Android SDK** - Development kit

5.2 Project Structure

```
Bettr_Mobile_App/
├── app/src/main/
│   ├── java/com/bettr/
│   │   ├── Activities/
│   │   │   ├── Login.java       # Login screen
│   │   │   ├── Register.java    # Registration screen
│   │   │   ├── Feed.java        # Main feed
│   │   │   └── Intro.java       # Welcome screen
│   │   ├── Fragments/           # UI fragments
│   │   ├── Adapters/            # List adapters
│   │   ├── ApiRest/
│   │   │   ├── Api_Gets.java    # GET requests
│   │   │   └── Api_Inserts.java# POST requests
│   │   ├── Dao/
│   │   │   └── Connection.java # Local connection
│   │   ├── Models/              # Data models
│   │   └── UserSession.java     # Session management
│   └── res/                     # Resources (layouts, strings, etc.)
└── build.gradle                 # Build configuration
```

---

6. Desktop Application

6.1 Technology Used

- **C#** - Programming language
- **Windows Forms** - GUI framework
- **.NET Framework 4.7.2** - Development platform
- **HttpClient** - HTTP client for API

6.2 Project Structure

```
Bettr_Desktop_App/
├── Bettr/
│   ├── Forms/
│   │   ├── Login.cs          # Login screen
│   │   ├── Register.cs       # Registration screen
│   │   ├── Dashboard.cs      # Main dashboard
│   │   └── CompleteProfile.java# Complete profile
│   ├── Api/
│   │   └── ApiService.cs     # API service
│   ├── Models/
│   │   ├── User.cs           # User model
│   │   └── Habit.cs          # Habit model
│   └── App.config            # Configuration
└── Bettr.sln                 # Visual Studio solution
```

---

7. Security

7.1 Security Mechanisms

Password Hashing

- **BCrypt** is used to hash passwords before storing
- Passwords are never sent in plain text

Injection Protection

- **SQL Injection:** The API uses prepared statements for all database queries
- **XSS (Cross-Site Scripting):** The web application sanitizes all user inputs

Data Validation

- All data received by the API is validated before processing
- Field types, formats, and lengths are verified

7.2 User and Permission Management

- Each user has a unique ID
- Users can only modify their own profile
- Follow/unfollow operations require authentication
- Habits can only be deleted by their creator

7.3 Implemented Best Practices

- Passwords hashed with BCrypt
- Input validation on all endpoints
- Clear separation between sensitive and non-sensitive data

---

8. Deployment

8.1 How to Run Each Application

Android Application

1. Open Android Studio
2. Import the `Bettr_Mobile_App` project
3. Run on an emulator or physical device

Desktop Application

1. Open the `Bettr.sln` solution in Visual Studio
2. Build and run (F5)

8.2 API Deployment

The API is deployed on **Render** (cloud hosting service).

Production URL: `https://bettr-g5yv.onrender.com/rest`

Note: Render's free plan has limitations:

- The API may take 5-10 seconds to respond after a period of inactivity
- After the first request, it responds faster

Steps to deploy on Render:

1. Create an account on render.com
2. Connect the GitHub repository
3. Select the API project
4. Configure environment variables
5. Deploy

8.3 Environments

Environment	URL	Description
Development	localhost:8080	Running locally
Production	bettr-g5yv.onrender.com	API deployed in the cloud

---

9. Testing

9.1 Unit Tests

Formal unit tests are not implemented in the project. Below are the manual tests performed:

User Tests

- [x] Register new user
- [x] Login with valid credentials
- [x] Login error with invalid credentials
- [x] Update user profile
- [x] Search users

Habit Tests

- [x] Create new habit
- [x] View habit feed
- [x] View specific user’s habits
- [x] Like a habit
- [x] Unlike a habit

Follower Tests

- [x] Follow a user
- [x] Unfollow a user

- [x] View followers list
- [x] View following list

9.2 Other Tests

- **Integration tests:** Verification that all three applications communicate correctly with the API
- **Security tests:** Verification that passwords are hashed
- **Performance tests:** Verification of API response time

---

10. Usage and Maintenance Guide

10.1 Running Locally

Prerequisites

- Java JDK 11+
- Maven
- Android Studio (for mobile app)
- Visual Studio (for desktop app)
- PHP Server (XAMPP/WAMP)

Steps to run

2. **Web:** Copy files to a PHP server
3. **Android:** Import in Android Studio and run
4. **Desktop:** Open solution in Visual Studio and run

10.2 Adding Features

To add new features:

1. **API:** Create new endpoints in the corresponding package
2. **Database:** Add new tables or fields if necessary

3. **Apps:** Implement the interface in each application

10.3 Common Problems

Problem	Solution
API not responding	Verify the server is running
Connection error	Check the API URL in configuration
Render API is slow	Normal on free plan; wait a few seconds
Login error	Verify credentials and internet connection
Android compilation error	Clean project and sync Gradle

---

11. Future Improvements

11.1 Planned Improvements

- Implementation of push notifications
- Messaging system between users
- API performance improvement

11.2 New Features

- **Habit comments:** Allow commenting on other users' habits
- **Hashtags:** Tag system for categorizing habits
- **Statistics:** Charts and habit statistics
- **Achievements:** Achievement system for completing habits

Appendix: Application URLs

Resource	URL
Base API	https://bettr-g5yv.onrender.com/rest
Users Endpoint	https://bettr-g5yv.onrender.com/rest/users
Habits Endpoint	https://bettr-g5yv.onrender.com/rest/habits